Open Access | https://doi.org/10.25080/gewj5574



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

ngff-zarr: A Lean and Kind OME-Zarr Toolkit for Bioimaging

Matt McCormick[1], **Johannes Soltwedel**[2], **Wouter-Michiel Vierdag**[3,4], **Valentin Boussot**[1], and **Josh Moore**[2]

[1]Fideus Labs LLC, [2]German BioImaging — Society for Microscopy and Image Analysis e.V., [3]Cellonautica Inc, [4]scverse

Abstract

Modern bioimaging instruments produce datasets that are large, multidimensional, and stored in vendor-specific proprietary formats. These monolithic files are not cloud-ready, are difficult to stream or share, and hinder reproducible, collaborative science. The community has responded with OME-Zarr (OME-NGFF), an open, chunked, cloud-native standard built on Zarr's compressed, n-dimensional array storage. We present `ngff-zarr`, a lean, minimal-dependency Python toolkit that implements the OME-Zarr specification and makes it accessible through a simple, four-step pipeline interface. `ngff-zarr` converts, validates, and generates multiscale representations of extremely large images out-of-core via Dask, accepting any input that follows the Python Array API Standard (NumPy, Dask, CuPy, PyTorch). It provides multiple downscaling methods, including SIMD-accelerated Gaussian filtering via ITK-Wasm; OME-Zarr Zip (`.ozx`) single-file archives (RFC-9); RFC-4 anatomical orientation metadata for medical and neuroimaging interoperability; emerging RFC-5 coordinate systems and transformations with OME-Zarr version 0.6; high-content screening plate and well support; TIFF, OME-TIFF, and Leica LIF conversion; and Zarr v3 sharding. We further describe the `ngff-zarr-mcp` Model Context Protocol (MCP) server, which exposes conversion, inspection, validation, and optimization tools to AI agents so that researchers can perform bioimaging tasks in natural language. We close by discussing lessons learned, planned support for additional community RFCs, and the path toward OME-Zarr 1.0.

Keywords OME-Zarr, OME-NGFF, bioimaging, cloud-native, Dask, Model Context Protocol

Published Jul 13, 2026

Correspondence to
Matt McCormick
matt@fideus.io

Open Access

Copyright © 2026 McCormick *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Modern bioimaging instruments, including light-sheet microscopes, high-content screening systems, whole-slide scanners, and electron or volume-electron microscopes, routinely produce datasets that are large, multidimensional, and stored in fragmented, vendor-specific proprietary formats. A single acquisition can exceed the memory of any workstation, and the resulting monolithic files are not cloud-ready: they are difficult to stream, to share, and to align with FAIR (Findable, Accessible, Interoperable, Reusable) principles. This friction directly hinders reproducible, collaborative science.

The bioimaging community has converged on an answer. OME-Zarr — created by the OME-NGFF, Open Microscopy Environment Next-Generation File Format, community — is a community-driven, open standard for storing bioimaging data in the cloud [1], [2]. Built on Zarr's chunked, compressed, n-dimensional array storage [3], OME-Zarr stores images as multiscale pyramids together with rich, machine-readable metadata. Its specification and ecosystem are developed openly through a Request for Comments (RFC) process and periodic community hackathons [4]. Adoption now spans diverse modalities and institutions worldwide.

A specification, however, only delivers value when backed by robust, accessible tooling. This is the gap that motivates `ngff-zarr` [5], a lean, minimal-dependency Python implementation of OME-Zarr that is lazy, parallel, and web-ready. `ngff-zarr` is designed around two goals captured in its tagline, *lean and kind*: lean in that it adds little beyond the scientific Python stack it builds on, and kind in that its interface maps to how researchers actually think about their data, including a recently added natural-language interface for AI agents.

This paper makes three contributions. First, we describe the design and implementation of `ngff-zarr`: an out-of-core, Array-API-based pipeline that turns in-memory arrays into validated, multiscale OME-Zarr stores. Second, we catalog the toolkit's features and how they serve concrete bioimaging needs, from anatomical orientation for medical imaging to high-content screening and the emerging RFC-5 coordinate-transformation support targeting OME-Zarr 0.6. Third, we present the `ngff-zarr-mcp` Model Context Protocol server and report lessons learned from exposing scientific tooling to AI agents and from working within the OME-Zarr community.

1.1. Background: OME-Zarr and the community

Rather than a single monolithic file, an OME-Zarr dataset is a hierarchy of chunked, compressed arrays accompanied by JSON metadata [6]. Three properties make it well suited to modern data. It is **chunked and compressed**, so a client can read a small region of a large image without downloading the whole dataset — the foundation of cloud-optimized access. It is **multiscale**, storing each image as a singular scale or as a pyramid of progressively downsampled resolutions for responsive visualization and scale-appropriate analysis. And it is **self-describing**: axis names, types, units, and coordinate transformations are stored alongside the data in a standardized model shared across the community's many implementations and programming languages.

The format's design and the community process behind it are documented in J. Moore *et al.* [1] and J. Moore *et al.* [2], and the ecosystem continues to evolve through open RFCs and collaborative events such as the 2024 OME-NGFF workflows hackathon [4]. `ngff-zarr` is developed within and for this community, tracking the specification as it advances from version 0.1 through 0.5, and now toward 0.6, alongside the Zarr Format Specification 3.

2. Methods

`ngff-zarr` is implemented in Python (requiring Python 3.10 or newer) with a deliberately small dependency footprint built on NumPy [7], Dask [8], Zarr [3], and ITK-Wasm [9]. Its interface provides an abstraction layer between front-end usage and the terms of the spec, to provide users with a stable interface and a backend that is able to evolve and forward metadata as the specification and its terms evolve. This backend uses Python `dataclasses` to represent the specification model directly which is accessible to users for inspection, along with convenient access to the pixel data as lazy Dask arrays. The core abstraction is a four-step pipeline: an in-memory array becomes an `NgffImage`, which becomes a multiscale `NgffMultiscales`, which is written to an OME-Zarr store.

```python
import ngff_zarr as nz

image = nz.to_ngff_image(
    array,
    dims=["z", "y", "x"],
    scale={"z": 2.0, "y": 0.5, "x": 0.5},
)
multiscales = nz.to_multiscales(image, scale_factors=[2, 4], chunks=64)
nz.to_ngff_zarr("output.ome.zarr", multiscales)
```

2.1. *Array to `NgffImage`*

The pipeline accepts any array-like object that follows the Python Array API Standard [10], including NumPy `ndarrays`, Dask arrays, Zarr arrays, CuPy arrays, and PyTorch tensors. The `to_ngff_image` function wraps the array in an `NgffImage`, a plain `dataclass` representing a single scale level. When constructing the image, the user may specify the dimension names `dims` drawn from `{'t', 'z', 'y', 'x', 'c'}`, the physical pixel `scale` for the spatial dimensions, a `translation` giving the origin of the first pixel, a `name`, and `axes_units` expressed with UDUNITS-2 (standard for physical units) identifiers. Sensible NumPy-compatible defaults are used for anything left unspecified. Internally the data is held lazily as a chunked Dask array, so no pixels are loaded or computed until they are needed.

2.2. *`NgffImage` to `NgffMultiscales`*

`to_multiscales` builds a Dask task graph that will produce a chunked, multiscale image pyramid. It accepts optional `scale_factors` and `chunks` parameters and an antialiasing `method`. The returned `NgffMultiscales` dataclass holds the image for each scale together with the OME-Zarr metadata — axes, datasets, and coordinate transformations — and the correct `scale` and `translation` for each downsampled level are computed automatically. Because the result is still a lazy task graph, building the multiscales is inexpensive; the work happens only when the store is written.

2.3. *Downscaling methods*

Generating a faithful multiscale pyramid requires care: to avoid aliasing artifacts, the input must be smoothed before each downsampling step. `ngff-zarr` exposes a family of downscaling methods through the `ngff_zarr.Methods` enumeration, summarized in Table 1. They differ primarily in their smoothing strategy and in their suitability for intensity versus label images, trading off artifact level, speed, and hardware portability.

Table 1. *Downscaling methods available via `ngff_zarr.Methods`.*

Method	Backend	Image type
`ITKWASM_GAUSSIAN` (default)	ITK-Wasm	Intensity
`ITKWASM_BIN_SHRINK`	ITK-Wasm	Intensity
`ITKWASM_LABEL_IMAGE`	ITK-Wasm	Label
`ITK_GAUSSIAN`	Native ITK	Intensity
`ITK_BIN_SHRINK`	Native ITK	Intensity
`DASK_IMAGE_GAUSSIAN`	dask-image / SciPy	Intensity
`DASK_IMAGE_MODE`	dask-image / SciPy	Label
`DASK_IMAGE_NEAREST`	dask-image / SciPy	Label

The default method, `ITKWASM_GAUSSIAN`, smooths with a discrete Gaussian filter to build a proper scale space, ideal for intensity images. It is implemented with ITK-Wasm [9], so it

is SIMD-accelerated and extremely portable across platforms, including the browser. The availability of label-aware methods is important: naively downsampling a segmentation with a smoothing filter corrupts label identities, so dedicated mode-based reductions (`ITKWASM_LABEL_IMAGE`, `DASK_IMAGE_MODE`) are provided. Native ITK variants [11] and SciPy-backed `dask-image` fallbacks round out the set for environments with different dependency or hardware constraints.

2.4. `NgffMultiscales` to OME-Zarr store, reading, and validation

`to_ngff_zarr` computes the pyramid and writes it to a Zarr store. Because every stage of the pipeline is expressed as a Dask task graph, the entire computation executes lazily and out-of-core: chunks are streamed through the graph and written incrementally rather than materialized all at once. This is what allows `ngff-zarr` to process datasets that exceed available memory. By convention a local directory store uses the `.ome.zarr` extension, but any Zarr store type may be used — including remote object stores on S3, Google Cloud Storage, or Azure via `fsspec`, with no local filesystem required. OME-Zarr version 0.4 (Zarr Format Specification 2) and 0.5 (Zarr Format Specification 3) are supported on write, and versions 0.1 through 0.5 can be read.

Reading mirrors writing. `from_ngff_zarr` returns an `NgffMultiscales` populated with lazy Dask arrays. With the optional `validate` dependency installed, passing `validate=True` checks that the store's metadata conforms to the shared OME-Zarr JSON schema model, raising an error on any deviation; validation is supported for versions 0.1 through 0.5.

```
multiscales = nz.from_ngff_zarr("cthead1.ome.zarr", validate=True)
```

3. Results

The result of this design is a small toolkit that nonetheless covers a broad range of real bioimaging workflows. We describe its principal features and the needs they address.

3.1. Out-of-core conversion of extremely large data

The central capability is converting arbitrarily large images into validated, multiscale OME-Zarr without exhausting memory. Because the pipeline is built on Dask task graphs, a multi-hundred-gigabyte volume can be converted on a laptop: chunks flow through smoothing, downsampling, and writing stages incrementally. The command-line interface (CLI) makes this explicit through a `--memory-target` option that accepts a human-readable limit and schedules the computation to respect it.

```
ngff-zarr --memory-target 50M -i LIDCFull.vtk -o LIDCFull.ome.zarr
```

3.2. OME-Zarr Zip (`.ozx`) single-file archives

A directory-based Zarr store can comprise thousands of small files, which is awkward to copy, share, or archive. RFC-9 [12] introduces the OME-Zarr Zip format, which packages an entire OME-Zarr hierarchy into a single ZIP archive with the `.ozx` extension. `ngff-zarr` reads and writes `.ozx` transparently — the extension is detected automatically — and embeds the OME-Zarr version in the ZIP comment for reliable detection on read. For large datasets, `write_store_to_zip` copies an existing store directly into a `.ozx` archive without recomputing arrays.

```
nz.to_ngff_zarr("cthead1.ozx", multiscales, version="0.5")
multiscales = nz.from_ngff_zarr("cthead1.ozx")
```

3.3. RFC-4 anatomical orientation

Medical and neuroimaging analysis depends on knowing the anatomical direction of each spatial axis so that images can be aligned to atlases and to one another. RFC-4 [13] adds anatomical orientation metadata to OME-NGFF axes. `ngff-zarr` emits this metadata when RFC-4 is enabled, either programmatically via `enabled_rfcs=[4]` in `to_ngff_zarr` or with the `--enable-rfc 4` flag on the command line. When converting ITK or ITK-Wasm images — for example from NRRD, NIfTI, or DICOM inputs — anatomical orientation is derived automatically from ITK's LPS (Left-Posterior-Superior) coordinate system [11], with convenience constants provided for both the LPS and the neuroimaging RAS conventions.

```
nz.to_ngff_zarr("output.ome.zarr", multiscales, enabled_rfcs=[4])
```

3.4. RFC-5 coordinate systems and OME-Zarr 0.6

The most significant recent addition is emerging support for RFC-5, which provides first-class coordinate systems and transformations in OME-Zarr and is the centerpiece of version 0.6 [14]. RFC-5 introduces named coordinate systems (sets of axes) and a richer vocabulary of transformations — including identity, axis permutation and projection, translation, scale, affine, rotation, sequences of transformations, and field-based displacement and coordinate transforms — that map points between coordinate systems. This enables datasets to express the spatial relationships between multiple images, such as aligned tiles or registered modalities, in a standardized, machine-readable way without resampling and re-saving pixel data.

`ngff-zarr` implements RFC-5 toward OME-Zarr 0.6, including coordinate-system handling and transformation sequences during multiscale generation, an updated 0.6 JSON schema set, and round-trip conversion between versions 0.5 and 0.6. RFC-5 is designed to work hand in hand with RFC-4: where anatomical axes are not aligned with imaging axes, RFC-5 transformations describe the relationship between image space and anatomical space while RFC-4 supplies the biological orientation labels.

3.5. High-Content Screening

High-content screening (HCS) produces multi-well plate data central to drug discovery and high-throughput imaging. `ngff-zarr` implements the OME-Zarr plate and well metadata structures, so plates can be loaded, navigated by row and column, and examined field by field, with optional acquisition (time point or condition) selection and HCS-aware validation.

```
plate = nz.from_hcs_zarr("screening_plate.ome.zarr")
well = plate.get_well("A", "1")   # row A, column 1
image = well.get_image(0)         # first field of view
print(image.images[0].data.shape)
```

3.6. Format conversion, sharding, and the CLI

Through optional dependencies, the toolkit converts a broad range of scientific image formats to OME-Zarr: any format readable by ITK, `tifffile`, or `imageio`, including TIFF and multi-series OME-TIFF with automatic extraction of physical pixel sizes and units, as well as Leica LIF microscopy files. For very large outputs, Zarr v3 sharding stores multiple

compressed chunks in a single file or blob, reducing file counts; sharding is requested with the `chunks_per_shard` argument and requires OME-Zarr version 0.5 or newer. An optional Tensorstore backend can further improve write performance. For batch and scripting workflows, the CLI mirrors the library — a basic conversion is a single command, and omitting the output prints information about the input and the multiscales that would be generated.

```
ngff-zarr -i input.nrrd -o output.ome.zarr
```

3.7. MCP server for AI agents

The `ngff-zarr-mcp` package exposes the toolkit's capabilities to AI agents through the Model Context Protocol (MCP) [15], an open standard that lets AI models securely call external tools. Once configured, agents such as GitHub Copilot, Claude Code, Cursor, and OpenCode can drive `ngff-zarr` through natural language. The server provides a small, deliberate set of functions that map to researcher intent: `convert_images_to_ome_zarr` converts datasets with configurable parameters; `get_ome_zarr_info` reports detailed information about a store; `validate_ome_zarr_store` checks structure and metadata against the specification; and `optimize_ome_zarr_store` re-encodes an existing store with new compression or chunking. In practice a researcher might ask an assistant to *"convert `LIDCFull.tif` to OME-Zarr"*, *"find the optimal codec for this data"*, *"use sharding to keep the number of files under 20"*, or *"write a Python script to convert every file in this directory using that codec"*; the agent composes these tool calls and the server executes them.

4. Discussion

`ngff-zarr` demonstrates that a faithful, full-featured OME-Zarr implementation need not be heavy. By expressing the entire conversion pipeline as lazy Dask task graphs and by accepting any Array-API-compatible input, the toolkit keeps its dependency surface small while supporting workflows from a NumPy array in a notebook to a multi-terabyte light-sheet volume on cloud storage. The same design that enables out-of-core processing also makes the library portable: the default ITK-Wasm downscaling backend runs identically on a workstation, in CI, and in the browser.

4.1. Lessons learned from the MCP server

Designing agent-facing tools surfaced several lessons. First, **structured tool parameters matter**: clearly typed, well-described arguments make agent invocations far more reliable than free-form strings. Second, **functions should map to researcher intent, not to low-level API calls**: a single `optimize_ome_zarr_store` tool that an agent can reason about is more useful than exposing the full surface of internal functions. Third, **natural-language interfaces lower the barrier to adoption**: by letting scientists describe what they want rather than learn an API, the MCP server brings cloud-native formats and reproducible workflows to users who would otherwise be deterred by tooling.

4.2. Future work

Several directions are planned. On the specification side, we intend to extend support for additional community RFCs: **RFC-3** (support for additional dimensions) [16] for smoother interoperability with more image types, and **RFC-8** (collections) [17] for grouping related OME-Zarr datasets. We will continue maturing **RFC-5** coordinate-transformation support to directly support common use cases, and align releases with the community's **OME-Zarr 1.0** milestone — a stable, long-term-supported version of the format. Alongside these

features, we plan ongoing **performance improvements**, including faster downscaling and writing, better memory-aware scheduling, and expanded GPU acceleration. As an open-source project, `ngff-zarr` welcomes community contributions toward these goals.

5. Conclusion

`ngff-zarr` is a lean, minimal-dependency, and community-aligned implementation of the OME-Zarr specification. Its four-step pipeline — array to `NgffImage` to `NgffMultiscales` to store — gives researchers a simple, lazy, parallel, and web-ready path from in-memory data to a cloud-native bioimaging dataset, while out-of-core execution via Dask makes datasets larger than memory routine. Beyond the core pipeline, support for `.ozx` single-file archives, RFC-4 anatomical orientation, emerging RFC-5 coordinate transformations with OME-Zarr 0.6, High Content Screening, broad format conversion, Zarr v3 sharding, and a memory-aware command-line interface make the toolkit practical across medical imaging, microscopy, and high-throughput screening. The `ngff-zarr-mcp` server extends these capabilities to AI agents, demonstrating how natural-language interfaces can accelerate the adoption of FAIR, reproducible bioimaging workflows. `ngff-zarr` is open source under the MIT license, and contributions from the community are welcome.

5.1. Software availability

Source code is available at github.com/fideus-labs/ngff-zarr and documentation at ngff-zarr.readthedocs.io. The Python package is distributed on PyPI as `ngff-zarr` and the MCP server as `ngff-zarr-mcp`.

5.2. Acknowledgments

We thank the Open Microscopy Environment and the broader OME-Zarr community, including participants in the OME-NGFF workflows hackathons, for the open specification process and collaboration that make this work possible.

5.3. Disclosure of Gen AI use

Portions of this work were assisted using generative AI tools. The tools were used for drafting and refining text. All outputs were reviewed, verified, and revised by the authors, who take full responsibility for the accuracy and integrity of the final content.

References

[1] J. Moore *et al.*, "OME-NGFF: a next-generation file format for expanding bioimaging data-access strategies," *Nature Methods*, vol. 18, no. 12, pp. 1496–1498, 2021, doi: 10.1038/s41592-021-01326-w.

[2] J. Moore *et al.*, "OME-Zarr: a cloud-optimized bioimaging file format with international community support," *Histochemistry and Cell Biology*, vol. 160, no. 3, pp. 223–251, 2023, doi: 10.1007/s00418-023-02209-1.

[3] Zarr Developers, "zarr-python: An implementation of chunked, compressed, N-dimensional arrays for Python." [Online]. Available: https://zarr.dev/

[4] J. Lüthi and others, "2024 OME-NGFF workflows hackathon," *BioHackrXiv*, 2025, doi: 10.37044/osf.io/5uhwz_v2.

[5] M. McCormick and ngff-zarr contributors, "ngff-zarr: A lean and kind OME-Zarr implementation." [Online]. Available: https://github.com/fideus-labs/ngff-zarr

[6] Open Microscopy Environment Consortium, "Next-generation file format (NGFF) specifications for storing bioimaging data in the cloud." [Online]. Available: https://ngff.openmicroscopy.org/

[7] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[8] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," *Proceedings of the 14th Python in Science Conference*, pp. 130–136, 2015, doi: 10.25080/Majora-7b98e3ed-013.

[9] M. McCormick and P. Elliott, "ITK-Wasm," *Python in Science Conference*, 2024, doi: 10.25080/TCFJ5130.

[10] Consortium for Python Data API Standards, "Python array API standard." [Online]. Available: https://data-apis.org/array-api/latest/

[11] M. McCormick, X. Liu, J. Jomier, C. Marion, and L. Ibanez, "ITK: enabling reproducible research and open science," *Frontiers in Neuroinformatics*, vol. 8, p. 13, 2014, doi: 10.3389/fninf.2014.00013.

[12] Open Microscopy Environment Consortium, "RFC-9: OME-Zarr as a single file via ZIP." [Online]. Available: https://ngff.openmicroscopy.org/rfc/9/index.html

[13] Open Microscopy Environment Consortium, "RFC-4: Coordinate systems and transformations – anatomical orientation." [Online]. Available: https://ngff.openmicroscopy.org/rfc/4/index.html

[14] Open Microscopy Environment Consortium, "RFC-5: Coordinate systems and transformations." [Online]. Available: https://ngff.openmicroscopy.org/rfc/5/index.html

[15] Anthropic, "Model Context Protocol specification." [Online]. Available: https://modelcontextprotocol.io/

[16] Open Microscopy Environment Consortium, "RFC-3: Transitional bioformats2raw layout." [Online]. Available: https://ngff.openmicroscopy.org/rfc/3/index.html

[17] Open Microscopy Environment Consortium, "RFC-8: Collections." [Online]. Available: https://ngff.openmicroscopy.org/rfc/8/index.html